

MCU 8 点 FFT 官方兼容版

文件修改与 Vivado 操作手册

本手册基于远端 `speed_v5_clk95` 分支和课程“测试数据样例-2026”，用于完成官方 COE 格式兼容、Vivado 重建、仿真、实现和上板验收。

1. 仓库与基线

- 仓库：https://github.com/ENFP-Texas/MCU_FFT_baseline_pass
- 当前远端分支：`speed_v5_clk95`
- 速度版本标签：`speed_v5_cnt160_95MHz`
- 标签提交：`c91bcb6`
- 分支最新提交：`f90311e`
- FPGA：`xc7k160tffg676-2`
- Vivado：2025.1
- 当前结果：95 MHz、`cnt_test=160`、约 1.684 us
- 建议新分支：`official_compat_v1`

注意：远端分支名是 `speed_v5_clk95`，不是 `speed_v5_cnt160_95MHz`；后者是 tag。

2. 任务目标

使工程兼容课程提供的：

`/Users/rongzechen/资料/测试数据样例-2026.zip`

必须满足：

1. 使用完整 `FFT_input.coe` 作为 `test_ROM` 初始化数据。
2. 输出与 `FFT_output.coe` 的 16 个 16-bit word 完全一致。
3. FFT 的加减乘和三级蝶形仍由 MCU 指令完成，不能增加独立 FFT 硬件核。
4. `cnt_test` 从第一次读取 FFT 输入开始计数，在最后一次写 `verify_RAM[15]` 后停止。
5. 目标保持 `cnt_test=160`；若无法保持，必须解释原因并报告新数值。
6. 95 MHz 下实现后 WNS 必须为正。
7. 最终提交版移除 ILA/debug 资源及会阻止优化的 `mark_debug/keep`。

3. 官方文件格式

`FFT_input.coe` 是 144 个 16-bit word，不是 8-bit：

ROM 地址	内容	16-bit 定点格式
0~63	8×8 DFT 矩阵实部，按行排列	Q7
64~127	8×8 DFT 矩阵虚部，按行排列	Q7
128~135	x0~x7 实部	Q5
136~143	x0~x7 虚部	Q5

`FFT_output.coe` 是 16 个 16-bit Q12 word：

verify_RAM 地址	内容
0~7	X0~X7 实部
8~15	X0~X7 虚部

官方样例期望值：

```
addr 00..07 real:
f280 e80a 1a80 fea2 e080 16f6 e680 fa5e

addr 08..15 imag:
e900 317c 2c00 1f8c 0b00 0c84 1600 d874
```

直接使用 COE 中的 Q5 输入和 Q7 矩阵做整数复数矩阵乘法，已经确认与上述输出逐项一致。

4. 当前版本为什么不能直接通过

1. `test_ROM` 深度只有 16，地址只有 4 个有效位，无法访问 128~143。
2. 当前输入是 16-word 实虚交错；官方输入是实部区和虚部区分离。
3. 当前输出是实虚交错；官方输出要求 0~7 实部、8~15 虚部。
4. 当前常数是 Q15 的 `0x5A82`；官方矩阵使用 Q7 的 91/128。
5. 当前程序使用 `t=b*W; a±t`，但配对顺序是 DIF 顺序，不符合官方自然序输入。
6. 旧 Python 检查器复现了旧程序模型，不能作为官方结果判据。

5. 推荐的低风险定点方案

内部统一使用 Q12：

```
官方 Q5 输入 -> 读取接口符号扩展并左移 7 位 -> MCU 内部 Q12
```

保留当前 Q15 `MUL`，不要在兼容版中改 ALU。把旋转常数改成：

```
C707_OFFICIAL = 91 << 8 = 23296 = 0x5B00
```

现有 ALU：

```
(x * 0x5B00) >>> 15
```

与下面的 Q7 运算精确等价：

```
(x * 91) >>> 7
```

这样可以避免改动 `src/alu.v`、`src/control_unit.v`、`src/defines.vh` 和标准 `MUL` 指令语义。

6. 必须修改或新增的文件

6.1 输入 ROM 与地址链路

`ip/test_rom_dist/test_rom_dist.xci`

必须通过 Vivado Customize IP 修改，不建议手改 JSON：

```
Component name: test_rom_dist
Memory type: ROM
Data width: 16
Depth: 256
Address width: 自动生成 8
Input/output register: non-registered
Coefficient file: mem/FFT_input.coe
```

随后 Generate Output Products。

`src/test_rom_ip.v`

- `addr` 从 `[4:0]` 改为 `[7:0]`。
- IP 连接从 `.a(addr[3:0])` 改为 `.a(addr)`。
- `test_vector_in` 保持 `[15:0]`。

- `INIT_FILE` 参数当前没有控制 XCI 初始化文件；可删除，或明确注释其无效。真正的初始化文件由 XCI 的 coefficient file 决定。

src/board_top.v

- `test_rom_addr` 从 [4:0] 改为 [7:0]。
- 修正仍写着 75 MHz 的过期注释，实际是 95 MHz。
- Debug 属性建议改成条件编译；最终版本不能始终保留 `mark_debug` 和 `keep`。
- 其他 verify 信号位宽不变。

src/mcu_top.v

- `test_rom_addr` 从 [4:0] 改为 [7:0]。
- 其余计数和 verify 端口不变。

src/mcu_core.v

- `test_rom_addr` 从 [4:0] 改为 [7:0]。
- ALU、控制器和 opcode 不改。

src/ext_test_rom_if.v

必须完成四项：

```
is_test_rom: TEST_BASE ~ TEST_BASE+255
test_rom_addr: mem_addr[7:0]
FFT 输入范围: TEST_BASE+128 ~ TEST_BASE+143
first_read_pulse: 第一次读 TEST_BASE+128
```

对 128~143 的 Q5 输入进行符号扩展并左移 7 位：

```
wire signed [31:0] raw_s16 =
    {{16{test_vector_in[15]}}, test_vector_in};

assign read_data = is_fft_sample
    ? (raw_s16 << 7)
    : raw_s16;
```

矩阵地址 0~127 保持普通符号扩展。FFT 程序可以使用固定旋转因子，不必读取整张矩阵。

6.2 FFT 指令程序

新增 `asm/fft8_official.asm`

不要覆盖旧汇编，保留回退版本。

初始化：

```
R0 = 0
R1 = 0
R6 = 0x5B00
R7 = TEST_BASE
```

第一层读取地址：

蝶形	a 实/虚	b 实/虚	W
(x0,x4)	128/136	132/140	W0
(x1,x5)	129/137	133/141	W1
(x2,x6)	130/138	134/142	W2
(x3,x7)	131/139	135/143	W3

每个蝶形必须改成正确 DIF：

```
sum = a + b
diff = a - b
upper = sum
lower = diff * W
```

旋转公式，`C=0x5B00`：

```
W0: tr=dr,      ti=di
W1: tr=(dr+di)*C,  ti=(di-dr)*C
```

```
W2: tr=di,      ti=-dr
W3: tr=(di-dr)*C,  ti=-(dr+di)*C
```

三级配对：

```
Stage 1: (0,4,W0) (1,5,W1) (2,6,W2) (3,7,W3)
Stage 2: (0,2,W0) (1,3,W2) (4,6,W0) (5,7,W2)
Stage 3: (0,1,W0) (2,3,W0) (4,5,W0) (6,7,W0)
```

该重排与现有程序使用相同数量的 load/add/sub/mul/store，目标仍是 165 条总指令、从首个 LDR 到最后 STR 为 160 个计数周期。

Stage 3 写入官方平面布局：

```
内部 data0 -> X0 -> verify real 0, imag 8
内部 data1 -> X4 -> verify real 4, imag 12
内部 data2 -> X2 -> verify real 2, imag 10
内部 data3 -> X6 -> verify real 6, imag 14
内部 data4 -> X1 -> verify real 1, imag 9
内部 data5 -> X5 -> verify real 5, imag 13
内部 data6 -> X3 -> verify real 3, imag 11
内部 data7 -> X7 -> verify real 7, imag 15
```

最后一条有效写操作必须是 `verify_RAM[15]`，然后 `HALT`。

重新生成

```
python3 scripts/assembler.py \
asm/fft8_official.asm \
-o mem/instr_fft8.mem \
--coe mem/instr_fft8.coe
```

检查汇编输出应约为：

```
assembled 165 instructions
```

src/instr_rom.v

删除硬编码路径：

```
E:/vivado/MCU/mem/instr_fft8.mem
```

改为可移植参数，默认文件名建议为：

```
instr_fft8.mem
```

并在 Vivado 中将 `mem/instr_fft8.mem` 添加为 Memory Initialization File。保留 `+INSTR_MEM=` 供仿真覆盖。

6.3 官方数据和验证脚本

新增：

```
mem/FFT_input.coe
mem/FFT_output.coe
scripts/fft_official_ref.py
scripts/check_fft_official.py
tb/tb_mcu_fft8_official.v
```

参考脚本必须：

1. 解析 COE 头和逗号/分号。
2. 确认输入恰好 144 words。
3. 将 0~127 解释为 Q7 矩阵，128~143 解释为 Q5 输入。
4. 直接用整数复数矩阵乘法生成 Q12 raw 结果：

```
Yr[k] = Σ (xr[n]*Mr[k,n] - xi[n]*Mi[k,n])
Yi[k] = Σ (xr[n]*Mi[k,n] + xi[n]*Mr[k,n])
```

因为 $Q5 \times Q7$ 正好得到 Q12，所以这里不再右移。

5. 按实部 0~7、虚部 8~15 排列。
6. 与 `FFT_output.coe` 和 RTL 输出逐项比较。
7. 非零退出码表示失败。

Testbench 至少检查：

```
16 个 verify 写地址均正确
16 个写数据与 FFT_output.coe 相同
最后写地址是 15
cnt_test 最终为 160
done 最终为 1
```

6.4 可选更新

- `scripts/disassemble_mem.py`：把常数注释从 `0x5A82` 更新/扩展为 `0x5B00`。
- `README.md`：说明官方格式、分支、测试命令和 Vivado 操作。
- `results/official_sample_result.md`：记录仿真、WNS、资源和上板结果。
- `src/ila_probe.v`：仅在 debug build 中增加 `test_rom_addr[7:0]`；最终版关闭。

7. 明确不需要修改的文件

兼容版优先不改：

```
src/alu.v
src/control_unit.v
src/decoder.v
src/defines.vh
src/reg_file.v
src/cnt_test.v
src/verify_ram_if.v
src/verify_ram_ip.v
constraints/top.xdc
ip/clk_wiz_0/clk_wiz_0.xci
```

其中 Clocking Wizard 必须继续保持：

```
输入 50 MHz
输出 95 MHz
```

`verify_ram_if.v` 已经在写地址 15 时产生 stop pulse，不需要修改，只需保证汇编最后写 15。

8. Vivado 2025.1 操作

8.1 工程可移植性

提交的 `vivado_project/MCU.xpr` 包含旧 Windows 绝对 ImportPath 和工程内部复制源文件。修改仓库 `src/` 后，旧 XPR 可能仍综合旧副本。

推荐新建干净工程：

1. Create Project, RTL Project。
2. Part 选择 `xc7k160tffg676-2`。
3. Add Design Sources：仓库 `src/*.v` 和 `src/defines.vh`，不要复制成另一套失联副本。
4. Add Constraints：`constraints/top.xdc`。
5. Add Memory Initialization File：`mem/instr_fft8.mem`。
6. Set as Top：`board_top`。
7. Add Simulation Sources：`tb/tb_mcu_fft8_official.v`。

若继续使用旧 XPR，必须先删除/替换 `MCU.srcs/.../imports/rtl` 中的旧源引用，并确认 Source Properties 的实际路径指向当前仓库。

8.2 Clocking Wizard

添加或重新加载：

```
ip/clk_wiz_0/clk_wiz_0.xci
```

Customize IP 核对：

```
Input clock: 50.000 MHz
Output clk_out1: 95.000 MHz
locked port: enabled
```

Generate Output Products。不要重新改回 50/75 MHz。

8.3 test_ROM IP

打开 IP Catalog -> Distributed Memory Generator, 或对现有 `test_rom_dist` 选择 Customize IP :

```
ROM
16-bit width
256 depth
non-registered output
COE = mem/FFT_input.coe
```

确认 Summary 中 :

```
C_WIDTH = 16
C_DEPTH = 256
C_ADDR_WIDTH = 8
```

然后 :

1. Reset Output Products。
2. Generate Output Products。
3. Report IP Status, 确保没有 locked/out-of-date IP。

现场替换随机 `FFT_input.coe` 后, 必须再次 Reset/Generate Output Products, 并 Reset Synthesis/Implementation 后重新生成 bitstream。

8.4 仿真

1. Simulation top 设为 `tb_mcu_fft8_official`。
2. Run Behavioral Simulation。
3. 观察 `test_rom_addr` 是否访问 0x80~0x8F。
4. 观察读入 MCU 的值是否等于 Q5 raw 左移 7。
5. 捕获 16 次 `verify_we`。
6. 比较官方输出。
7. 确认 `cnt_test=160`、`done=1`。

先通过仿真, 再运行综合。

8.5 综合与实现

首次兼容验证使用默认策略, 避免同时引入新的优化变量 :

```
Synthesis strategy: Vivado Synthesis Defaults
Implementation strategy: Vivado Implementation Defaults
Incremental compile: 首次关闭
```

操作 :

1. Reset Runs : `synth_1`, 同时重置 `impl_1`。
2. Run Synthesis。
3. Open Synthesized Design, 检查 `test_ROM` 地址宽度、层次和资源。
4. Run Implementation。
5. Report Timing Summary : 确认 95 MHz 时 WNS > 0。
6. Report Utilization : 记录 LUT、FF、DSP、BRAM。
7. Generate Bitstream。

Report Clocks 中应看到 :

```
CLK_50M period = 20.000 ns
clk_mcu period ≈ 10.526 ns
```

8.6 ILA 调试版

先做无 ILA 仿真。上板排错时再建立 debug build。

建议探针 :

```
test_rom_addr[7:0]
test_vector_in[15:0]
verify_addr[4:0]
verify_vector_out[15:0]
```

```
verify_we
cnt_test[19:0]
done
```

ILA depth 建议 1024。可在首次访问 `test_rom_addr==8'h80` 或 `done` 上升沿触发。

最终计分版本：

1. 删除 ILA 实例/debug hub。
2. 删除 debug.xdc 中自动生成的探针约束。
3. 关闭 `ENABLE_ILA`。
4. 移除或条件关闭 `mark_debug`、`keep`。
5. 重新综合、实现并记录最终资源和 WNS。

9. 验收标准

全部满足才算完成：

- [] 基于 `speed_v5_clk95` 新分支实现。
- [] ROM 为 256×16 ，地址宽度 8。
- [] 官方 144-word `FFT_input.coe` 可直接加载。
- [] 第一次 FFT 输入读取地址为 128。
- [] 输入 Q5 正确转换到内部 Q12。
- [] 使用正确 radix-2 DIF 蝶形。
- [] 旋转常数使用 `0x5B00`，保留现有 Q15 MUL。
- [] `verify_RAM[0:7]` 是 X0~X7 实部。
- [] `verify_RAM[8:15]` 是 X0~X7 虚部。
- [] 官方样例 16 words 全部精确匹配。
- [] `cnt_test=160` 并停止。
- [] `done=1`。
- [] 95 MHz 实现 $WNS > 0$ 。
- [] 至少 100 组随机 Q5 输入通过软件参考模型。
- [] 最终计分版本不含 ILA/debug 资源。
- [] 提交仿真日志、Timing Summary 和 Utilization 数据。

10. 实施顺序

严格按以下顺序工作：

1. checkout `speed_v5_clk95`，创建 `official_compat_v1`。
2. 添加官方 COE 和新的软件参考模型。
3. 先让参考模型复现官方 16-word 输出。
4. 修改 ROM IP 和 8-bit 地址链路。
5. 修改输入 Q5->Q12 适配。
6. 编写正确 DIF 汇编并重新生成指令 ROM。
7. 编写 testbench，完成官方样例 RTL 仿真。
8. 确认 `cnt_test=160`。
9. 建立/修复 Vivado 工程源文件引用。
10. 在 95 MHz 下综合、实现，确认 WNS 为正。
11. 上板 ILA 验证。
12. 删除 ILA/debug 后生成最终 bitstream。

不要在官方样例通过前进行多核、流水线、新 opcode 或乘法器资源优化。